

Automated Post-Trade Reconciliation & Exception Analytics Engine

Sara Patano, Alberto Tonni, Mehrdad Vafaei
Data-Intensive Computing

1 Overview

In post-trade processing, the two counterparties of a transaction each keep their own record of it, and small differences between those records, known as *trade breaks*, have to be found and resolved before settlement. In this project we built an automated engine that reconciles two such feeds and analyses the exceptions it finds. Starting from a public dataset of card transactions, we simulate two broker feeds, one clean and one with injected errors. The engine matches them record by record and labels each discrepancy by type. Records that cannot be compared are kept in a separate quarantine table together with the reason they were rejected. The pipeline runs on Apache Spark and is orchestrated by Apache Airflow. The results are loaded into Google BigQuery and explored in a Looker Studio dashboard.

2 Dataset

The pipeline starts from the *Financial Transactions Dataset* available on Kaggle. We use its `transactions_data.csv` file, which contains 13,305,915 card transactions with 12 attributes, and keep the six needed for matching and data-quality checks: `id`, `date`, `client_id`, `card_id`, `amount` and `merchant_id`. The `id` uniquely identifies a transaction, while the other fields record when it happened, the client and card involved, the amount and the merchant.

3 Data Preparation

Real bilateral feeds are not publicly available, so we simulate them from this dataset. Feed A keeps the records unchanged and is the reference. Feed B starts as a copy of A, and about 75% of its rows receive exactly one error, chosen at random among 20 types:

- realistic changes that should be reported as breaks: a shifted timestamp, or a different amount, client, card or merchant;
- small differences within tolerance (under a minute on the date, 0.01 on the amount), which should still count as matched;
- invalid values: missing (about 2% of rows), unparseable or outside the valid range;
- a changed transaction id (about 0.5% of rows), which leaves the record without a counterpart.

The generator draws a new seed and new weights for the error types on every run, so each run exercises a different mix of errors. The seed is saved together with the number of rows that should end up in each outcome, so any run can be reproduced, and the reconciliation step uses these counts to check its own results.

4 Methodology

Figure 1 shows the architecture of our pipeline, which runs locally with Docker Compose and sends its results to the cloud.

- **Apache Airflow** orchestrates the pipeline, starting each step only after the previous one has finished.
- **Feed generation** reads the source dataset and creates the two broker feeds: A as the clean reference, B with injected errors.

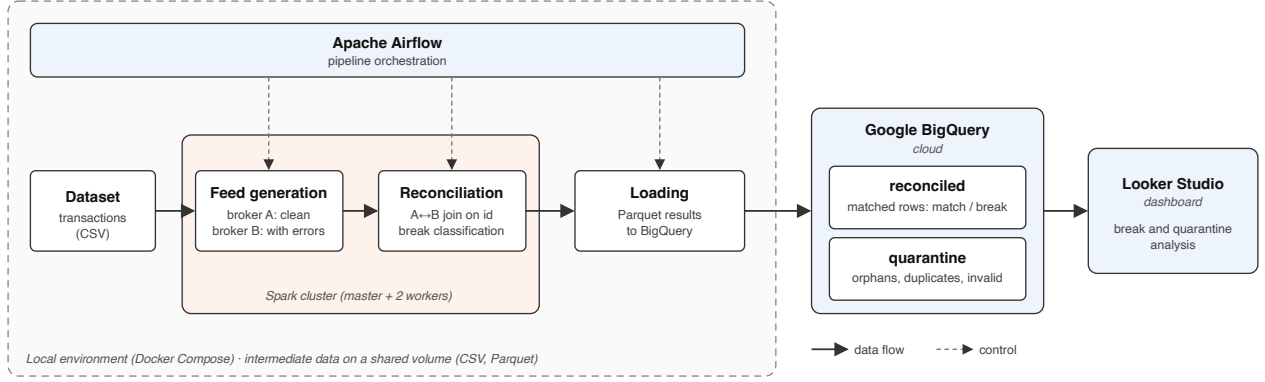


Figure 1: Pipeline architecture.

- **Reconciliation** validates every field and joins the two feeds on the transaction id. Each pair is labelled as matched or as a break on the field that differs, allowing up to 60 seconds on the date and 0.01 on the amount. Rows with invalid values, a duplicated id or no counterpart go to quarantine with the reason and the column that caused it.
- **Spark cluster** runs the two steps above, splitting the work between one master and two workers (2 cores and 2 GB each).
- **Loading** uploads the results to Google BigQuery.
- **Google BigQuery** stores them in two tables: **reconciled** for matched and broken pairs, **quarantine** for the rows set aside. Four SQL views (`bigquery/views.sql`) compute on top of them the exposure of each trade and the per-client and per-break-type aggregates used by the dashboard.
- **Looker Studio** reads the two tables and presents the breaks and quarantined rows in a dashboard.

4.1 Distributed implementation

Both feeds are read as plain strings and converted with Spark’s `try_cast` functions, which return null instead of failing on a bad value; this lets a single pass tell missing values (`NULL_VALUE`) from malformed ones (`UNPARSEABLE`) and from values outside the valid range (`OUT_OF_DOMAIN`). Duplicate ids are detected with a window function, `count(*)` over a window partitioned by `id`, so no separate aggregation and self-join is needed. Rows whose id is unusable are removed before the join, and the remaining rows of A and B are combined with a `FULL OUTER JOIN` on `id`: a missing side becomes `MISSING_IN_A` or `MISSING_IN_B`. Spark executes the join as a shuffle on the join key, so rows with the same id end up in the same partition and the work is spread across both workers. The break checks are plain column expressions evaluated in the same stage, without user-defined functions, so Spark can optimise the whole plan. Because the joined result feeds two outputs (reconciled and quarantine), it is persisted to disk once instead of being recomputed; we use `DISK_ONLY` storage to stay within the 2 GB of memory of each worker. Each output is written as a single Parquet file, which BigQuery loads in one job. In BigQuery, **reconciled** is partitioned by month and clustered by **status**, and **quarantine** is clustered by **reason**, so the dashboard queries scan only the blocks they need.

5 Results

For every run, the reconciliation job compares the number of rows in each status and quarantine reason with the counts recorded by the generator, and fails if any of them differs. Since a failed check stops the DAG before the loading step, every run that reaches the dashboard has passed it. On our local machine, the whole pipeline on the 13.3 million rows takes about 12 minutes, from starting the containers to the updated dashboard.

The results are presented in a Looker Studio dashboard (Figure 2). The top row shows the main KPIs:

total reconciled pairs, matched pairs, breaks and total exposure, defined as the sum over all pairs of the absolute difference between the amounts reported by A and B. Below, the dashboard shows the ten clients with the highest exposure, the distribution of break types, and a table of the quarantined rows with the reason and the column that caused them.

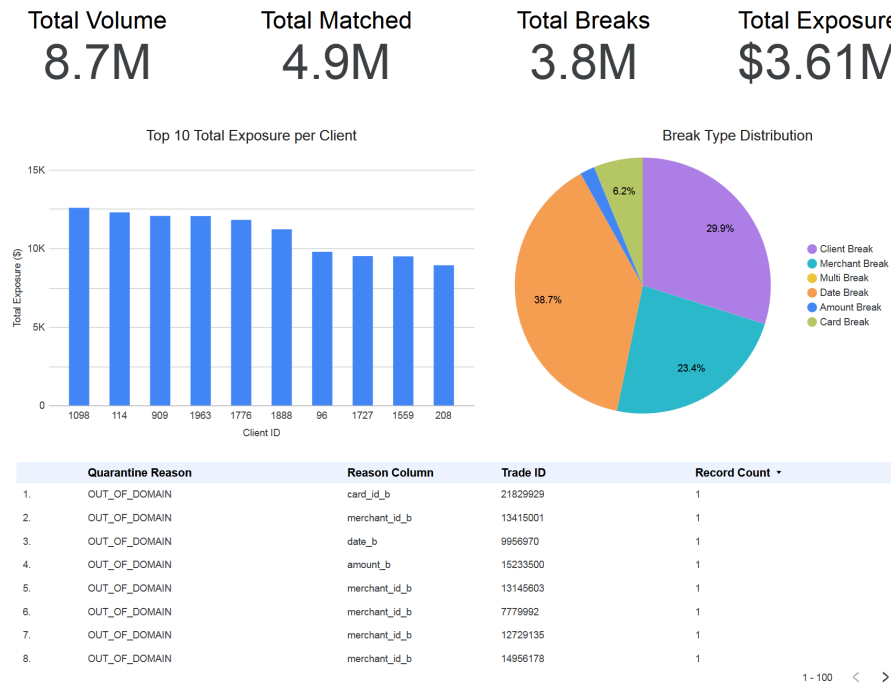


Figure 2: Dashboard produced in Looker Studio.

6 Limitations and Future Work

The feeds are simulated, and every row of B carries at most one error, so the `MULTI_BREAK` status is never produced; real feeds would also need fuzzy matching when ids differ. Intermediate data lives on a shared Docker volume rather than on HDFS, which is enough for a single machine but would be replaced by HDFS or Cloud Storage on a real cluster. The pipeline runs as a batch on demand; the next step is a daily schedule fed by market data APIs, and moving the BigQuery views into dbt models with automated data tests.

7 Running the Code

Requirements: Docker, and a Google Cloud project with BigQuery and a service account key.

Setup

1. Place the dataset in `data/source/transactions_data-selected-columns.csv`.
2. Place the service account key in `secrets/gcp-key.json`.
3. In the BigQuery console, run `bigquery/schema.sql` to create the tables, then `bigquery/views.sql` to create the views.

Run

4. Start the containers with `docker compose up -build`.
5. Open Airflow at `localhost:8081` and trigger the `trade_reconciliation` DAG. Spark progress is visible at `localhost:8080`.
6. Results appear in BigQuery under `trade_reconciliation_dev`, in the `reconciled` and `quarantine` tables.